

UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

SEMANA 04: ARQUITECTURA M-V-C

Documento similar de apoyo académico

Estudiante: Jordon Vera Madueño

Curso: Taller de Desarrollo de Aplicaciones

Ciudad: Huancayo

Año: 2026

Este material mantiene una estructura similar al PDF de la Semana 4: portada, enunciados, requerimientos funcionales, organización MVC y clases principales para los ejercicios 01 y 02.

Introducción

La arquitectura Modelo–Vista–Controlador (MVC) permite separar la lógica del negocio, la interfaz gráfica y la gestión de eventos dentro de una aplicación. Esta organización hace que el código sea más claro, reutilizable y fácil de mantener.

En este documento se presentan dos ejercicios desarrollados con una estructura similar a la guía de la Semana 4: un ejercicio de oferta de productos y un ejercicio de venta de pasajes, ambos organizados con clases en los paquetes MODELO, VISTA y CONTROLADOR, además de una clase principal para ejecutar cada caso.

Ejercicio 01 – Oferta de productos

1. Enunciado del proyecto

Una tienda ha puesto en oferta la venta de un producto ofreciendo un porcentaje de descuento sobre el importe de la compra de acuerdo con la cantidad de docenas adquiridas. Además, se entrega una cantidad de lapiceros de obsequio según el importe final por pagar.

2. Requerimientos funcionales

Nombre	R1: OfertaProducto
Resumen	Calcular el importe de la compra, descuento, importe a pagar y lapiceros de obsequio.
Entradas	Precio por docena y cantidad de docenas adquiridas.
Resultados	Importe de compra, importe del descuento, importe a pagar y cantidad de lapiceros.

3. Estructura MVC propuesta

- Paquete MODELO: clase **OfertaProducto**.
- Paquete VISTA: clase **FrmOferta** con los componentes gráficos.
- Paquete CONTROLADOR: clase **ControladorOferta** para capturar eventos y actualizar la vista.
- Paquete principal: clase **Ejer1_Sem4** para iniciar la aplicación.

4. Clases principales

MODELO / OfertaProducto.java

```
package MODELO;
public class OfertaProducto {
    private double precioDocena;
    private int cantidadDocenas;
    public double calcularImporteCompra(){ return precioDocena * cantidadDocenas; }
    public double calcularDescuento(){ return (cantidadDocenas >= 10) ? 0.20 : 0.10; }
    public double calcularImporteDescuento(){ return calcularImporteCompra() *
    calcularDescuento(); }
    public double calcularImportePagar(){ return calcularImporteCompra() -
    calcularImporteDescuento(); }
    public int calcularLapiceros(){ return (calcularImportePagar() >= 200) ? 2 *
    cantidadDocenas : cantidadDocenas; }
}
```

VISTA / FrmOferta.java

```
package VISTA;
public class FrmOferta extends JFrame {
    private JTextField txtPrecio;
    private JTextField txtCantidad;
```

```
private JButton btnCalcular;  
private JTextArea txtResultado;  
}
```

CONTROLADOR / ControladorOferta.java

```
package CONTROLADOR;  
public class ControladorOferta implements ActionListener {  
private OfertaProducto modelo;  
private FrmOferta vista;  
public void actionPerformed(ActionEvent e){  
// leer datos, calcular y mostrar resultados  
}  
}
```

Main / Ejer1_Sem4.java

```
package com.mycompany.ejer1_sem4;  
public class Ejer1_Sem4 {  
public static void main(String[] args){  
OfertaProducto modelo = new OfertaProducto();  
FrmOferta vista = new FrmOferta();  
ControladorOferta controlador = new ControladorOferta(modelo, vista);  
controlador.iniciar();  
}  
}
```

Ejercicio 02 – Venta de pasajes

1. Enunciado del proyecto

Una empresa de transportes cubre la ruta Lima–Huancayo en dos turnos: mañana y noche. Según la cantidad de pasajes adquiridos se aplica un porcentaje de descuento, y además se entregan caramelos de obsequio en función del importe final por pagar.

2. Requerimientos funcionales

Nombre	R1: VentaPasaje
Resumen	Calcular el importe de la compra, descuento, importe a pagar y caramelos de obsequio.
Entradas	Turno seleccionado y cantidad de pasajes.
Resultados	Importe de compra, importe del descuento, importe a pagar y cantidad de caramelos.

3. Estructura MVC propuesta

- Paquete MODELO: clase **VentaPasaje**.
- Paquete VISTA: clase **FrmVenta** con un combo de turnos y una caja para cantidad.
- Paquete CONTROLADOR: clase **ControladorVenta** para responder al botón Calcular.
- Paquete principal: clase **Ejer2_Sem4** para iniciar el ejercicio.

4. Clases principales

MODELO / VentaPasaje.java

```
package MODELO;
public class VentaPasaje {
    private String turno;
    private int cantidadPasajes;
    public double getPrecioPasaje(){ return 37.5; }
    public double calcularImporteCompra(){ return getPrecioPasaje() * cantidadPasajes; }
    public double calcularDescuento(){ return (cantidadPasajes > 15) ? 0.08 : 0.05; }
    public double calcularImporteDescuento(){ return calcularImporteCompra() *
    calcularDescuento(); }
    public double calcularImportePagar(){ return calcularImporteCompra() -
    calcularImporteDescuento(); }
    public int calcularCaramelos(){ return (calcularImportePagar() > 200) ? 2 *
    cantidadPasajes : cantidadPasajes; }
}
```

VISTA / FrmVenta.java

```
package VISTA;
public class FrmVenta extends JFrame {
    private JComboBox cboTurno;
    private JTextField txtCantidad;
    private JButton btnCalcular;
    private JTextArea txtResultado;
}
```

CONTROLADOR / ControladorVenta.java

```
package CONTROLADOR;
public class ControladorVenta implements ActionListener {
    private VentaPasaje modelo;
    private FrmVenta vista;
    public void actionPerformed(ActionEvent e){
        // leer turno, calcular valores y mostrar boleta
    }
}
```

Main / Ejer2_Sem4.java

```
package com.mycompany.ejer2_sem4;
public class Ejer2_Sem4 {
    public static void main(String[] args){
        VentaPasaje modelo = new VentaPasaje();
        FrmVenta vista = new FrmVenta();
        ControladorVenta controlador = new ControladorVenta(modelo, vista);
        controlador.iniciar();
    }
}
```

Conclusión

La arquitectura MVC mejora la organización interna de una aplicación al separar responsabilidades. En ambos ejercicios, el modelo concentra los cálculos, la vista administra los componentes visuales y el controlador coordina la interacción del usuario con la lógica del programa. Esta forma de trabajo facilita la comprensión del código y el mantenimiento del proyecto académico.